



Update at Your Own Risk: Analysis and Recommendations for Update-Related Vulnerabilities

Ahmad B. Usman^(✉)  and Mikael Asplund 

Department of Computer and Information Science, Linköping University, Linköping, Sweden

{ahmad.usman,mikael.asplund}@liu.se

Abstract. This paper presents an empirical analysis of software and firmware update vulnerabilities in computing systems, resulting in a set of recommendations for implementing more secure update mechanisms. Using a Common Vulnerabilities and Exposures (CVE) dataset, we analyze trends over the past eight years, focusing on the frequency and impact of these vulnerabilities on confidentiality, integrity, and availability, as well as their associated attack vectors and severity levels. We identify distinctive patterns compared to the broader vulnerability dataset, offering insights for risk assessment and management. Our findings reveal that vulnerabilities in software and firmware update (and upgrade) processes have a greater impact than the average disclosed vulnerability. Our results also indicate that the primary attack vector for update-related vulnerabilities is local, whereas for overall vulnerabilities, exploitation typically occurs over the network. Furthermore, we investigate the most common weakness classifications associated with these vulnerabilities, identifying scenarios that illustrate their adverse effects on systems and what they enable an attacker to achieve. Common weaknesses among update-related vulnerabilities include improper verification of cryptographic signatures, improper certificate validation, and improper input validation. From the top most prevalent weaknesses, we systematically derive a set of recommendations to help mitigate or eliminate attacks and breaches that occur during the update process.

Keywords: Updates · Vulnerabilities · Empirical analysis · Mitigations

1 Introduction

Modern computer-based systems, including IoT devices, are vital in sectors such as healthcare, transportation, electric charging systems and critical infrastructure, where they provide enhanced functionality and real-time monitoring capabilities. The core functionality of these devices is determined by their software and firmware, and a key feature of such devices is their ability to receive software or firmware updates. This capability is meant to provide security, longevity,

and adaptability to new requirements. Without regular updates, software and firmware can become outdated and more susceptible to cyber-attacks, necessitating periodic updates to mitigate risks and address known security flaws that attackers could exploit. The updates can be applied automatically or manually and are often delivered remotely (potentially over the air).

Unfortunately, many update mechanisms are misconfigured, and challenges exist in securely distributing updates [10]. Even worse, many software update mechanisms lack adequate security controls [1, 5], and sometimes vendors are not adhering to the best security practices [3]. Vulnerabilities in the update mechanism can lead to unauthorized access, and data breaches, posing significant risks to the overall security of the IoT and CPS environments. In July 2024, a failed CrowdStrike update caused a global outage, leading to widespread Windows system crashes with the Blue Screen of Death (BSOD) error. This failure disrupted airports, healthcare facilities, and organizations worldwide.

This paper aims to empirically investigate the vulnerabilities and weaknesses associated with software and firmware updates, either due to misconfigurations in the update process or due to update mechanism failures. We examine vulnerability reports related to the update process, including metrics such as the level of the confidentiality, integrity, and availability of systems and the likelihood of their exploitation. We consider these numbers in comparison to the overall vulnerabilities reported in the last eight years. This study also aims to provide a deeper understanding of the security risks and the most common weaknesses (CWE) associated with the software and firmware updates. This allows us to develop a recommendation matrix to mitigate against the potential causes in the systems. In summary, the following are the contributions of this paper:

- We conduct an empirical analysis of vulnerabilities in the update process, identifying key differences and comparing them to general sets of all types of vulnerabilities (Sect. 3.1).
- We analyze real-world vulnerabilities reported in the CVE database and highlight illustrative cases where security breaches occurred due to update-related weaknesses (Sect. 3.2).
- We develop a recommendation matrix to mitigate or eliminate the root causes of update-related weaknesses (Sect. 4).

2 Methodology

Research Questions: To meet the objectives of this empirical study, we aim to address the following **Research Questions (RQs)**:

- **RQ1:** How do software and firmware update vulnerabilities differ from other vulnerabilities in terms of their impact on confidentiality, integrity, and availability, the likelihood and methods of exploitation, and their severity level?
- **RQ2:** What are the most common weaknesses in update processes?
- **RQ3:** Based on the characteristics of previously reported weaknesses, what mitigation strategies should be prioritized for update mechanisms?

Data Collection: We primarily collected data from the CVE List¹ a resource maintained by MITRE.

We downloaded vulnerability incidents disclosed by the official CVE organization See Footnote 1 and selected those from the past eight years as shown in Fig. 1, as recent vulnerabilities contain more precise scoring metrics and detailed information compared to older ones.

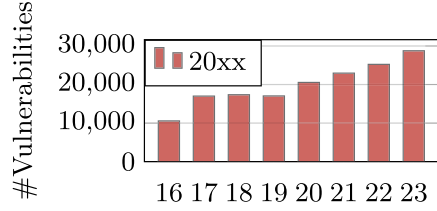


Fig. 1. Overall vulnerabilities.

Extracting Update-related Vulnerabilities: For a vulnerability to be considered as related to the update process, it must involve a defect or malicious activities occurring in the following scenarios: (i) exploiting a flaw while applying an update (e.g., man-in-the-middle attacks during update uploading/-downloading). (ii) compromising the integrity of the update itself (e.g., providing malicious updates or tampering with update mechanisms). (iii) vulnerabilities related to the upgrade or downgrade process. (iv) a vulnerability in the logic or permissions that govern how the update is executed (e.g., weak update management). The vulnerability retrieval and query generation in our study are performed as a semi-automated process. To curate our dataset, we first downloaded the vulnerabilities as a JSON format and selected data spanning the years 2016 to 2023. Following this, we manually performed an extensive filtering process on the initial dataset of $n = 159,463$ vulnerabilities, as illustrated in Fig. 2. Our algorithm searched for the keywords "software updat*", "firmware updat*", "software upgrad*", "firmware upgrad*", "updat* process", "upgrad* process", "downgrad*", "over-the-air," and "OTA." This search yielded exactly $n = 1,613$ vulnerabilities excluding overlaps. This process is shown in Fig. 2. We identified $n = 422$ software update-related and $n = 236$ firmware-related update vulnerabilities after cleaning, reviewing and refining the dataset.

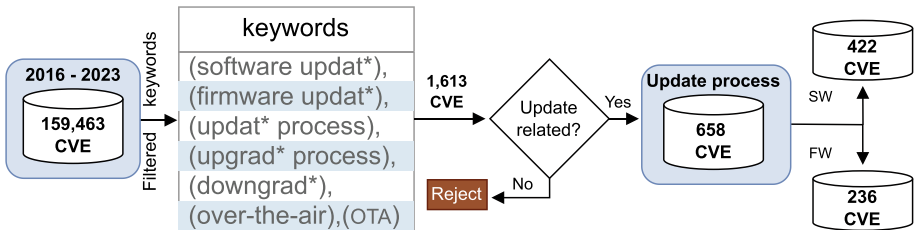


Fig. 2. Dataset curation process of update-related vulnerabilities.

Figure 3 shows vulnerabilities in the update process. The histogram indicates that update-related vulnerabilities have been increasing in recent years

¹ <https://www.cve.org/downloads>.

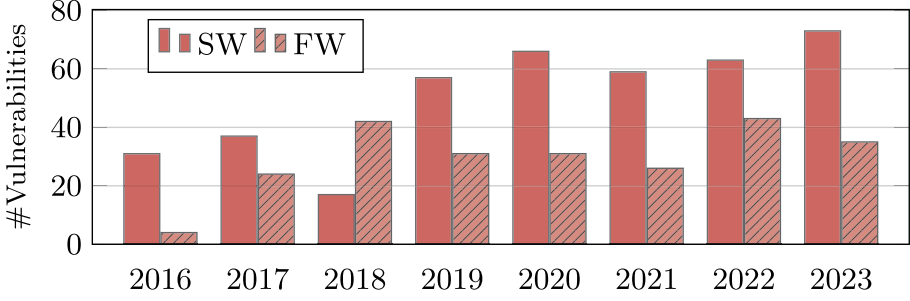


Fig. 3. Update-related vulnerabilities statistic in the last eight years.

Analysis of Selected Metrics: We derived CVSS metrics to account for the frequency of impact on confidentiality, integrity, availability, attack vector, attack complexity, and the severity level. To achieve this, we let $\mathcal{D}$ represent the dataset containing V vulnerabilities, $\mathcal{D} = \{CVE_1, CVE_2, \dots, CVE_V\}$, where each CVE_i contains the vulnerability label metrics in the dataset. For each CVE_i the impact metrics, access metrics and severity metrics are represented by the following sets:

- **Impact metrics:** $\{C_i, I_i, A_i\}$, for confidentiality, integrity and availability.
- **Access metrics:** $\{AV_i, AC_i\}$, for access vector, and access complexity.
- **Severity metrics:** $\{S_i\}$, where $\{S_i\}$ is the severity level.

The process iterates over all the $CVE_V \in \mathcal{D}$, extracts the details and counts the occurrences of each metric value. Let $Freq(X)$ denote the frequency of counter for a given metric X , the cumulative counters can be expressed as:

$$Freq(X) = \sum_{i=1}^V \mathbb{I}(X_i), \quad (1)$$

where V is the total vulnerabilities, and $\mathbb{I}(X)$ is an indicator function that counts accumulation of a specific value for a the given metric X in each CVE_V .

Mapping CVE to CWE Algorithm: We computed the number of CVEs associated with each CWE in the dataset. Let I represent the set of all CWEs and J represent the set of all CVEs. For a given CWE $i \in I$, let N_i denote the number of CVEs associated with it. This can formally be expressed as:

$$N_i = \sum_{j \in J} e_{ij} \quad \text{where} \quad e_{ij} = \begin{cases} 1, & \text{if CVE } j \text{ maps to CWE } i \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

3 Analysis of Vulnerabilities and Weaknesses

This section analyzes update process vulnerabilities (CVEs) to address **RQ1** and determine whether they differ significantly from the overall set of vulnerabilities.

In addition, this section identifies the most common weaknesses (CWEs) to address **RQ2** and explains how these weaknesses can be exploited during an update process, providing illustrative samples in which such attacks might occur.

3.1 Update Process Vulnerability Analysis (RQ1)

We analyzed the vulnerabilities in the update process to understand their impact, the methods used for exploitation with regards to the attack vector, the complexity of these exploits, and the severity of the vulnerabilities.

The upper part of Fig. 4 illustrates the empirical distribution of vulnerabilities in update processes in terms of exploitability, impact and severity, while the bottom presents the empirical distribution of overall vulnerabilities, allowing us to observe how they may differ. Regarding the impact on confidentiality, integrity, and availability, high-impact violations are more prevalent in both categories compared to low or no impact. However, the proportion of high-impact vulnerabilities is larger for update process vulnerabilities compared to the overall dataset. Moreover, the exploitability analysis shows that local attack vectors are more common in update process vulnerabilities than network-based (i.e., remotely), adjacent network, or physical access attacks. In contrast, overall vulnerabilities are more frequently exploited through network-based attacks. Finally, the base severity level of update process vulnerabilities is higher than that of overall vulnerabilities in both the critical and high severity categories.

Table 1 presents the proportion of update process vulnerabilities compared to overall vulnerabilities. The fully red color represents a higher percentage in update process vulnerabilities, whereas the muted red represents a lower percentage in the overall dataset. We can conclude that the impact and severity of update process are higher compared to overall recent years.

Table 1. Comparison of impact violation, exploitability and the base severity proportion between update process vs. overall vulnerabilities.

	Update process			Overall		
	High	Low	None	High	Low	None
Confidentiality	65.78%	7.98%	26.24%	47.83%	31.74%	26.26%
Integrity	72.24%	14.07%	13.69%	40.49%	35.26%	30.07%
Availability	73.38%	6.84%	19.77%	44.08%	21.71%	40.03%
	Update process			Overall		
	Critical	High	Medium	Critical	High	Medium
Severity	11.79%	49.05%	36.12%	8.06%	37.59%	48.80%

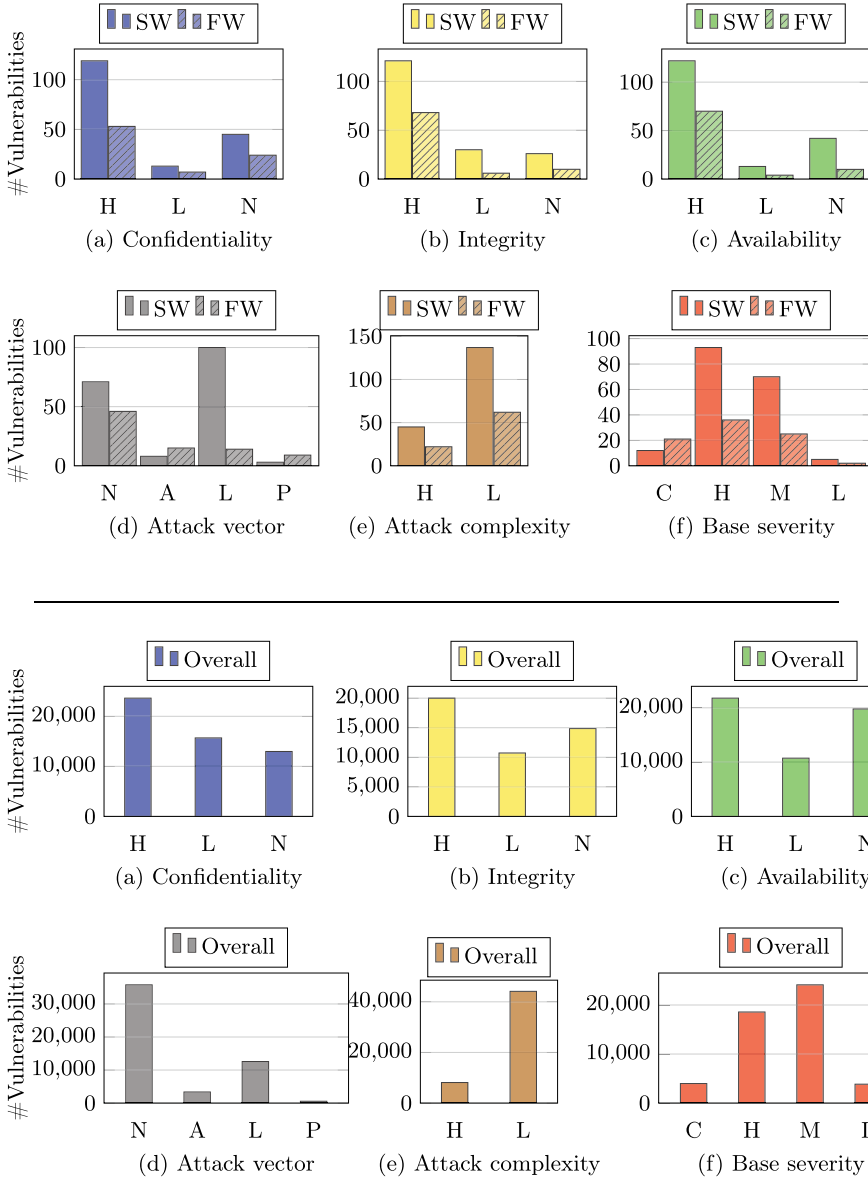


Fig. 4. Impact, exploitability and severity of software (SW), firmware (FW) and overall vulnerabilities: (a,b,c,e,f)=H:High, M:Medium, L:Low, N:None, C:Critical. (d)=N:Network, A:Adjacent network, L:Local, P:Physical.

3.2 Most Common Update Process Weaknesses (RQ2)

By analyzing the metric fields, for mapping the CVE to CWE (cf. Sect. 2) of the vulnerability data we identified the most common of the reported weaknesses related to the update process. Table 2 provides the top ten most frequently exploited weaknesses in update processes, along with their overall occurrence in vulnerabilities. Identical CWEs share identical colors to visually highlight their appearance in several places in the table.

Table 2. Top ten update process and overall weaknesses.

SW		FW		SW+FW		Overall	
CWE-ID	Prop.	CWE-ID	Prop.	CWE-ID	Prop.	CWE-ID	Prop.
1 CWE-347	5.03%	CWE-347	12.19%	CWE-347	7.47%	CWE-79	14.32%
2 CWE-295	4.40%	CWE-121	7.31%	CWE-345	4.98%	CWE-20	5.00%
3 CWE-22	4.40%	CWE-345	7.31%	CWE-20	4.98%	CWE-89	4.87%
4 CWE-345	3.77%	CWE-20	7.31%	CWE-284	4.56%	CWE-352	3.62%
5 CWE-284	3.77%	CWE-284	6.09%	CWE-427	4.15%	CWE-125	3.49%
6 CWE-427	3.77%	CWE-755	6.09%	CWE-295	3.32%	CWE-200	3.08%
7 CWE-378	3.77%	CWE-427	4.87%	CWE-494	3.32%	CWE-22	2.83%
8 CWE-20	3.77%	CWE-367	3.65%	CWE-22	3.32%	CWE-416	2.75%
9 CWE-306	3.14%	CWE-494	3.65%	CWE-120	2.49%	CWE-787	2.59%
10 CWE-494	3.14%	CWE-120	3.65%	CWE-121	2.49%	CWE-284	2.55%
Missing	62.32%	Missing	65.25%	Missing	63.37%	Missing	72.05%

From Table 2, and by combining software and firmware weaknesses, it is evident that improper verification of cryptographic signatures, insufficient verification of data authenticity, and improper input validation are the most prevalent issues in the update process. Meanwhile, Cross-Site Scripting (XSS) and SQL injection are more commonly associated with overall vulnerabilities.

To structure our findings, we group (high-to-low) closely-related weaknesses into six categories (ARW, PRW, ORW, IIV, IAC, ICV) and provide a brief description based on the MITRE definitions, followed by a contextual explanation of how these weaknesses specifically arise during the update process. Finally, we include real-world examples from our curated dataset to illustrate the potential capabilities these vulnerabilities provide to attackers.

ARW (Authentication-related Weaknesses): Improper Verification of Cryptographic Signature (CWE-347**) , Insufficient Verification of Data**

Authenticity (CWE-345) , and Download of Code Without Integrity Check (CWE-494): These weaknesses arise when the product fails to adequately verify the authenticity or origin of update data. This may include failing to verify cryptographic signatures or executing unverified remote code. In the context of updates, these weaknesses are typically manifested when the device does not at all, or incorrectly, verify the authenticity of the update binary. This could be concerning the update download, or before deploying the update from a file already on disk. E.g., in CVE-2023-4589, the digital signature is missing and update integrity is not verified during update process, or in CVE-2016-6567, the download manager fails to verify the firmware authenticity before executing the code, allowing an attacker to inject malicious update, replace entire OS or execute arbitrary code with root privileges.

PRW (Path control-related Weaknesses): Uncontrolled Search Path Element (CWE-427) and Improper Limitation of a Pathname to a Restricted Directory (CWE-22): The product builds a pathname using external input to locate a directory within a restricted parent directory but fails special elements neutralization , or the product uses a fixed search path for resources, but parts of the path may be controlled by unauthorized actors. In the context of updates, the device might not verify that update download destinations are within the intended directory tree due to uncontrolled search path or insufficient filtering of element. There are typically multiple pathways through which update information is accessed, one of which could be under the control of an attacker. E.g., in CVE-2020-8144, the web-interface does not validate the destination of the downloaded firmware in its update functionality, and in CVE-2023-32660, the firmware update tool has an uncontrolled search path, allowing an attacker to manipulate the file path and escalate privileges.

ORW (Overflow-related Weaknesses): Buffer Copy without Checking Size of Input (CWE-120) and Stack-based Buffer Overflow (CWE-121): The product copies data from an input buffer to an output buffer without checking if its size fits or the product allows its allocated buffer on the stack to be overwritten. In the context of updates, this can occur if the device accepts updates that exceed the intended size and writes beyond the allocated space. E.g., in CVE-2021-21967, the over-the-air update functionalities is vulnerable to out-of-bounds write, or e.g., in CVE-2023-35748, the server of the firmware processing over-the-air updates vulnerable to stack-based buffer overflow, allowing an attacker perform man-in-the-middle attack, a DoS attack

IIV (Improper Input Validation) (CWE-20): The product does not validate the input it received correctly. In the context of updates, the update mechanism does not correctly validate the raw-data input such as parameters, strings, or the metadata input of the firmware image or files during upload or download of updates. E.g., in CVE-2017-6759, the update mechanism (UpgradeManager) is vulnerable due to insufficient input validation, manipulating the input functionality in the upgrade package, allowing an attacker to write arbitrary files.

IAC (Improper Access Control) (CWE-284): The product fails to restrict unauthorized actor to access the resource properly. In the context of updates, the update mechanism or content can be easily accessible by the attacker due to weak or missing credentials. E.g., in CVE-2023-28070, the device is improperly accessible during update process, allowing an attacker to escalate privileges.

ICV (Improper Certificate Validation) (CWE-295): The product fails to validate a certificate correctly. In the context of updates, the certificates for the updates is not validated. E.g., in CVE-2022-3913, when downloading an updates, the server fails to validate the certificates, allowing an attacker to escalates privilege. This is somewhat similar to authentication-related weaknesses; however, we address it separately because certificates require special management especially in terms of mitigation using remote attestation.

3.3 Observation and Limitations

The lack of sufficient CVSS metrics negatively impacted our conclusions regarding both update-related and overall vulnerability impacts.

Since this study relies on the CVE dataset, we face the following limitations:

- **Missing Metrics Labeling:** When analyzing the impact and exploitability, we were only able to process 41.69%, 36.02%, and 31.06% of the $n = 422$, $n = 236$ and $n = 159,038$ software, firmware and overall vulnerabilities, respectively, due to the CVE dataset not assigning metric labels to the CVEs.
- **Missing CWE labeling:** Similarly, the CVE dataset does not associate the CWEs associated for the majority of CVEs.
- **Erroneous Classification:** Some of the CVEs seem to be incorrectly classified in the dataset, for instance, the CVE-2021-25437 is categorized as an Improper Input Validation (CWE-20), however, based on the description it seems that this should be classified as Improper Access Control (CWE-284).
- **CWE Abstraction:** Some CWEs are highly abstracted with weakness descriptions that are often generic in context and broad in applicability.

Moreover, there is likely an under-reporting of CVEs generally that also affects our results. We only analyzed 37.67%, 34.74% and 27.95 for software, firmware and overall vulnerabilities respectively. Despite these limitations, our results provide an initial guide for prioritizing mitigation strategies.

4 Recommendations for Secure Update (RQ3)

In this section, we present recommendations for mitigating risks associated with update processes, leading us to answer **RQ3**.

Mitigation Recommendations: Taken together, the top 10 weaknesses correspond to 41.08% of the reported update-related vulnerabilities that have CWE information. Despite the probable under-reporting of vulnerabilities and the lack

of CWE-information in the majority of CVEs, this number indicates that mitigating these particular weaknesses would have a significant impact of the overall security of update processes. Therefore, we identify six generic mitigation recommendations that can be used as a checklist when designing, implementing, or deploying an update mechanism. We base these recommendations on two main sources, (i) MITRE's recommended mitigations and (ii) techniques and solutions described in scientific literature. MITRE provides recommended mitigations for most of the CWEs. These mitigations are of varying specificity and applicability in the context of updates. For each of the top 10 weaknesses, we analyze the set of recommended mitigation listed by MITRE and assess their suitability for inclusion in our proposed list. The criteria we have used in this selection were (i) the mitigation should have specific relevance for update-related vulnerabilities since otherwise we would end up with a list of generic security advice, (ii) the mitigation should be sufficiently concrete to be implementable when designing update mechanisms, and (iii) there should be a sufficiently high potential effect of the mitigation on at least one of the weaknesses.

This analysis resulted in four distinct mitigation recommendations (R1-R4). In addition to the MITRE recommendations, we considered whether there were something missing based on what has been described in literature in relation to secure updates. This resulted in two recommendations (R5-R6). Note that we make no claim that our proposed list of recommendations is complete. There could be other highly relevant mitigation recommendations that complement the ones we describe. Below we provide the mitigation recommendations (R1-R6) derived from MITRE and the literature.

R1: Automated Static Analysis (ASA); a method for analyzing the binary code before they are executed on the device. According to MITRE, ASA is typically performed by building a control-flow analysis and searching for violated paths that connect the input origin with the destination. Therefore, ASA can detect potential software update issues like coding violations and runtime errors [14]. Automated dynamic analysis can also be effective in the case of firmware [4]. By combining both approaches, we can further achieve good soundness and completeness [15].

R2: Sandbox Environment; a method used by which unsafe software code can be run in an isolated environment, in case that the code is malicious, it will not affect other local applications, network resources or the host OS. In the context of updates, implementing a sandbox or jail environment ensures that updates are executed in a restricted area and have been rectified [7], enforcing boundaries with the existing version of the update or the live environment before deployment on the device. However, the limitation in mitigation's effectiveness relies on the sandbox's capabilities and may only reduce the attack's scope.

R3: Input Validation; by assuming all inputs are malicious, we can deploy input validation strategies to enforce restrictions and reject inputs that do not comply with the process's requirements. In the context of update, validating all the user input accessing the raw-data and metadata of the updates including

the formats, sizes, string lengths and numeric values are within predefined limits and comply with the device specification.

R4: Environment Hardening; is a strategy that emphasizes strengthening the security of an environment by minimizing the privileges granted to processes or code. In the context of update, this is done by running the code with the lowest privileges required to perform the updates, for example, we can restrict the rights to access directory or hard-coded file paths leading to update content.

R5: Remote Attestation; is a security mechanism where a trusted third party (verifier) assesses the integrity of a system (prover) [2,9,13], by verifying its state, typically through cryptographic proofs. It is widely used even in complex systems, decentralized systems or IoT devices [3]. More importantly in relation to our studies, this technique has been integrated, to ensure secure software updates in embedded systems [12], connected vehicles and memory overflow protection in firmware, software or cloud environment [2].

R6: Update Framework; there are dedicated software frameworks designed to supply software or firmware update reliably and securely. A prominent example that has received both academic and industrial attention is The Update Framework (TUF), which can be integrated into most update systems, it can also provide protection in the case that the system has been compromised by an attacker. Another update framework is Uptane [8], which is based on TUF but adapted for the automotive industry. Furthermore, Uptane can combine verification of updates with remote attestation [8].

Mapping Recommendations to Weaknesses: Once established, we mapped the six recommendations to the weaknesses it addresses, which results in a recommendation matrix. To indicate the potential effectiveness of each recommendation, we consider both the prevention and detection effectiveness ratings from MITRE². It categorizes *prevention* rates (EffectivenessEnumeration) as high, moderate, limited, defense in depth, incidental, or discouraged, and detection rates (DetectionEffectivenessEnumeration) as high, moderate, SOAR partial, opportunistic, limited, or none. We opt for a more course-grained classification as we do not have sufficient amounts of data for a fine-grained analysis of the effectiveness of each mitigation. Therefore, in our rating, we classify a measure as “*prevents (P)*” or “*detects (D)*” if there is evidence that the mitigation is effective for a given weakness, and as “*partially prevent (PP)*” or “*partially detect (PD)*” if there is evidence for some or limited effectiveness. In some cases we find no evidence or indication that the mitigation is effective to counteract some weakness, and then we leave the field empty. Table 3 illustrates the mapping of update process weaknesses to the recommendation matrix (based on their relevance in our dataset). A single * indicates that the recommendation is explicitly provided either by MITRE, or in the literature, but not its effectiveness, while a double ** signifies that both the recommendation and its rating are explicitly provided by the source. Here we motivate our decision for rating matrix that is not rated by MITRE (i.e. single *).

² <http://cwe.mitre.org/documents/schema/index.html>.

Table 3. Mitigation recommendation matrix for update-related weaknesses:

Weakness	R1	R2	R3	R4	R5	R6
ARW	D**	PP**		PP**	D	P
PRW		PP**		PP*	PD	PP
ORW	D**	PP**	P**	PP**	PD	PP**
IIV	D**		P**		D	PP*
IAC					PD	
ICV	PD**				PD	

Note: ARW (Authentication-related CWEs), PRW (Path-related CWEs), ORW (Overflow-related CWEs), IIV (Improper Input Validation), IAC (Improper Access Control), ICV (Improper Certificate Validation).

Environment hardening can mitigate path-related weaknesses (R4*PRW), as highlighted by MITRE³. However, we classify this as a partial prevention mitigation because reducing privileges to access specific paths or directories cannot fully prevent certain types of attacks. For instance, in some cases, a user can manipulate the path to point outside the hardened directories, thereby circumventing the protection mechanisms.

Remote attestation can partially detect malicious activities related to violations of access control (R5*IAC), path or directory exploitation (R5*PRW), and the acquisition of invalid certificates (R5*ICV) by verifying the integrity of update content, including access control policies and certificates, against the given baseline state. However, most current attestation techniques are static and may not detect violations occurring at runtime (i.e., during update process). Remote attestation can also partially detect improper input validation (R5*IIV) by monitoring log entries and user input activities, but it will be more effective if complemented with robust input validation techniques (R3). Furthermore, remote attestation can also be partially used to detect heap-based [2] or stack-based buffer overflows (R5*ORW).

The deployment of an **update framework** can partially prevent path-related and directory weaknesses (R6*PRW) [8], improper input validation (R6*IIV)⁴ [8], and can prevent against authentication-related weaknesses (R6*ARW) [8], since update frameworks, among other purposes, are specifically designed to ensure the authenticity of updates.

³ (R4*PRW): <https://cwe.mitre.org/data/definitions/22.html>.

⁴ (R6*IIV): <https://cwe.mitre.org/data/definitions/20.html>.

5 Related Work

To our knowledge, this is the first work that empirically analyzes software and firmware update vulnerabilities and identifies the most common weaknesses in computing systems. There are several works exploring vulnerabilities of software and firmware updates. The authors in [1] present disappointments and challenges with software updates, along with recommendations on how to carry out secure updates. However, their focus is mostly on operating systems, while also their recommendations were very generic rather than focused on addressing a particular weakness. The authors in [10] highlight the risks associated with software updates, particularly their delays, which leave systems vulnerable. However, they focus on addressing the challenges of investigating update behaviors due to the lack of vendor release schedules and the difficulty of tracking user update installations with data collection limited to three years. The authors in [11] present the impact of upgrading software vulnerabilities and its severity. Their primary aim is to propose a new method for assessing the vulnerability based on the CVSS. The authors in [6] conduct a security analysis of firmware updates on IoT devices. Their results indicate that most of these devices are vulnerable due to the adoption of insecure firmware update mechanisms. However, their analysis was limited to SDKs and IoT devices priced below \$50. Recently, Wu et al. [15] introduced a method to detect vulnerabilities in firmware updates. Through control-flow analysis, they identify and resolve the program execution paths during the update process. In relation to our work, the authors identified the top ten firmware update-related weaknesses, some of which overlap with our findings; however, their study is limited in scope to firmware vulnerabilities.

6 Conclusion

In this paper, we analyze the critical vulnerabilities associated with software and firmware updates in computing systems. The findings reveal that the severity of these vulnerabilities and their potential impact exceed those of overall reported vulnerabilities. We also found that the most common weaknesses in the update process are due to improper verification of cryptographic signatures, insufficient verification of data authenticity, and improper input validation. In contrast, common weaknesses across overall vulnerabilities include issues are due to XSS and SQL injection. We select representative cases from our curated dataset that demonstrate where these breaches can arise and how an adversary might exploit them. Finally, we developed a recommendation matrix to help reduce potential violations, assist system designers in prioritizing security measures, and mitigate update-related weaknesses that occur during the update process.

Acknowledgements. This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. The second author was partly financed by the Swedish Foundation for Strategic Research (SSF), project FUS21-0033 (ASTECC).

References

1. Bellissimo, A., Burgess, J., Fu, K.: Secure software updates: disappointments and new challenges. In: First USENIX Workshop on Hot Topics in Security (HotSec 06). USENIX Association (2006)
2. Boldyreva, A., Kim, T., Lipton, R., Warinschi, B.: Provably-secure remote memory attestation for heap overflow protection. In: Security and Cryptography for Networks. Springer (2016)
3. Bradley, C., Barrera, D.: Towards characterizing iot software update practices. In: Foundations and Practice of Security. Springer (2023)
4. Chen, D.D., Woo, M., Brumley, D., Egele, M.: Towards automated dynamic analysis for linux-based embedded firmware. In: (NDSS) (2016)
5. Coppens, B., De Sutter, B., De Bosschere, K.: Protecting your software updates. *IEEE Secur. Priv.* **11**(2), 47–54 (2013). <https://doi.org/10.1109/MSP.2012.113>
6. Ibrahim, M., Continella, A., Bianchi, A.: Aot - attack on things: a security analysis of iot firmware updates. In: 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P) (2023). <https://doi.org/10.1109/EuroSP57164.2023.00065>
7. Liu, S., Feng, P., Wang, S., Sun, K., Cao, J.: Enhancing malware analysis sandboxes with emulated user behavior. *Comput. Secur.* **115**, 102613 (2022). <https://doi.org/10.1016/j.cose.2022.102613>
8. Lorch, R., Larraz, D., Tinelli, C., Chowdhury, O.: A comprehensive, automated security analysis of the uptane automotive over-the-air update framework. In: Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses. (RAID '24), Association for Computing Machinery (2024). <https://doi.org/10.1145/3678890.3678927>
9. Plappert, C., Fuchs, A.: Secure and lightweight ecu attestations for resilient over-the-air updates in connected vehicles. In: ACSAC '23, Association for Computing Machinery (2023). <https://doi.org/10.1145/3627106.3627202>
10. Setayeshfar, O., Rhee, J.J., Kim, C.H., Lee, K.H.: Find my sloths: automated comparative analysis of how real enterprise computers keep up with the software update races. In: Detection of Intrusions and Malware, and Vulnerability Assessment. Springer (2021)
11. Treetippayaruk, S., Senivongse, T.: Security vulnerability assessment for software version upgrade. In: 2017 18th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD) (2017). <https://doi.org/10.1109/SNPD.2017.8022734>
12. Usman, A.B., Asplund, M.: Remote attestation with software updates in embedded systems. In: 2024 IEEE Conference on Communications and Network Security (CNS) (2024). <https://doi.org/10.1109/CNS62487.2024.10735526>
13. Usman, A.B., Cole, N., Asplund, M., Boeira, F., Vestlund, C.: Remote attestation assurance arguments for trusted execution environments. In: Proceedings of the 2023 ACM Workshop on Secure and Trustworthy Cyber-Physical Systems. (SaT-CPS '23), Association for Computing Machinery (2023). <https://doi.org/10.1145/3579988.3585056>
14. Wedyan, F., Alrmuny, D., Bieman, J.M.: The effectiveness of automated static analysis tools for fault detection and refactoring prediction. In: 2009 International Conference on Software Testing Verification and Validation (2009). <https://doi.org/10.1109/ICST.2009.21>
15. Wu, Y., et al.: Your firmware has arrived: a study of firmware update vulnerabilities. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association (2024)